

Asynchronous Processing with Message Queues

1. Asynchronous Workflows & Service Decoupling

To ensure sub-second response times during checkout, only core transactional actions (payment capture, seat reservation, order confirmation) run on the synchronous path. Non-blocking downstream operations are offloaded asynchronously via Apache Kafka:

Workflow	Producer	Kafka Topic	Consumer Service	Purpose
Ticket Notifications	BookingService	booking_events	NotificationService	Delivers booking confirmation and QR ticket via SMS and Email
Loyalty Rewards	BookingService	booking_events	LoyaltyService	Computes and credits reward points to user profile
Analytics Ingestion	BookingService	booking_events	AnalyticsService	Aggregates booking volume, revenue metrics, and demand logs

2. Producer and Consumer Roles

- Producer (BookingService):** Upon receiving successful payment verification and writing the confirmed booking record to PostgreSQL, BookingService publishes a BookingCompleted event to the booking_events topic.
- Consumer Groups:** Each downstream service operates under a distinct consumer group (notification-group, loyalty-group, analytics-group). This allows independent, parallel consumption without coupling or cascading slowdowns between notification delivery and analytics logging.

3. Retry Strategy and Dead-Letter Queues (DLQ)

External dependencies like SMS gateways and third-party email providers experience periodic network degradation. To prevent message loss or consumer stalling:

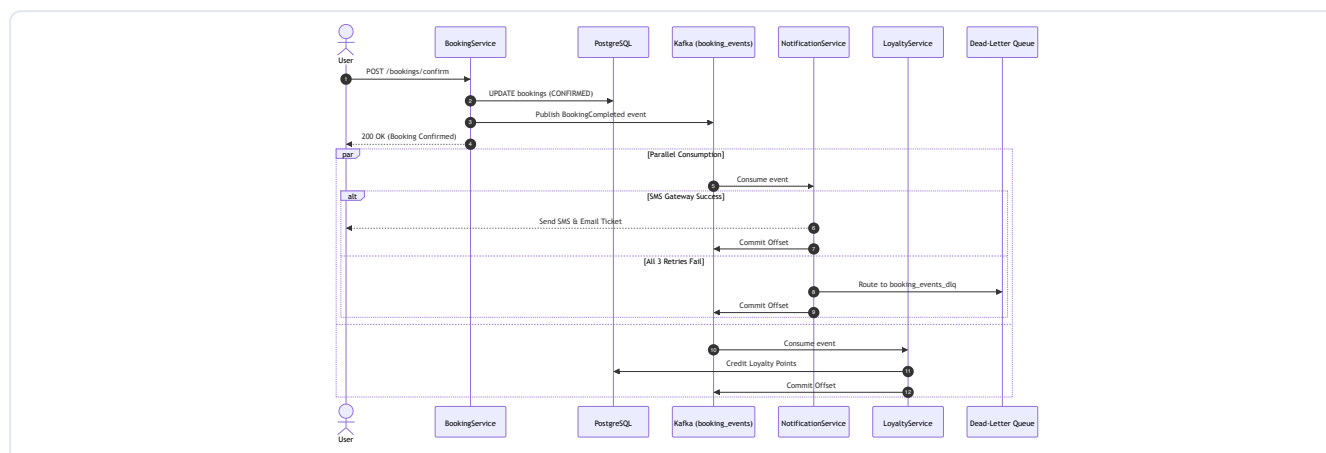
- Transient Retries:** When a consumer encounters a network error, it retries up to 3 times with exponential backoff (1s, 2s, 4s).
- Dead-Letter Routing:** If all retries fail, the message is routed to the dead-letter topic booking_events_dlq along with error metadata.
- Partition Non-Blocking:** Once forwarded to the DLQ, the consumer commits the offset on booking_events, allowing subsequent customer messages to process uninterrupted.

Consumer Retry & DLQ Logic

```
Algorithm: ProcessBookingEvent(event)
1. attempt = 0, max_retries = 3
2. while attempt < max_retries:
    try:
        ExecuteDelivery(event)
        AcknowledgeMessage(event)
        return
    catch NetworkException:
        attempt = attempt + 1
        Sleep(2 ^ attempt seconds)
3. PublishToDLQ("booking_events_dlq", event)
4. AcknowledgeMessage(event)
```

4. Sequence Diagram: Asynchronous Booking Flow

The sequence diagram below illustrates the end-to-end execution flow from synchronous confirmation to asynchronous event processing:



5. Message Idempotency & Exactly-Once Semantics

Because distributed message brokers operate with **at-least-once delivery**, network reconnects can trigger message re-delivery. To prevent duplicate side effects (e.g., sending duplicate SMS messages or double-crediting reward points), consumers enforce strict idempotency:

- Every event includes a unique `event_id` and `booking_id`.
- Each consumer service maintains a local `processed_events` table in its database with a unique primary key constraint on `event_id`.

Consumer Deduplication Logic

Algorithm: IdempotentConsumer(event)

```
1. exists = db.query("SELECT 1 FROM processed_events WHERE event_id = ?", event.event_id)
```

2. if exists:

AcknowledgeMessage(event)

return

3. ExecuteBusinessLogic(event)

```
4. db.execute("INSERT INTO processed_events (event_id, processed_at) VALUES (?, NOW())",
event.event_id)
```

5. AcknowledgeMessage(event)